Online Examination Assessment System Detailed Project Report

Prepared from the current repository state for viva and project presentation use.

This report summarizes the website, its architecture, major user journeys, database-backed auth flow, visual diagrams, and teammate contributions derived from the commit narrative.

Project type	Full stack online examination and assessment platform
Primary workflow	Exam creation, student attempt, grading, and result review
Frontend	Next.js application with dashboard and public site
Backend	Shared backend modules for auth, routing, and persistence
Database	PostgreSQL via Neon

The report is intentionally written in report style rather than commit log style. It focuses on the system as a product, then closes with equal contribution summaries for each teammate.

1. Project Overview

The Online Examination Assessment System is a role-based web application for managing the full lifecycle of online exams. It supports public landing pages, authenticated access, role-aware dashboards, question authoring, exam assembly, timed student attempts, grading, results, and audit-related operations.

The design goal of the project is operational clarity: each role sees only the controls and data they need, while the underlying services keep the exam state consistent and predictable. The application is not a marketing site; it is an operational assessment platform built for repeated use by administrators, examiners, and students.

The application is also designed as a presentation-ready system. The public site explains the product, the dashboard organizes work by role, and the data model supports real persistence instead of a fake in-memory demo. That makes the project suitable for both viva explanation and deployment review.

2. Website Structure and User Experience

The public side of the website introduces the platform with a hero section, a summary of capabilities, role cards, workflow highlights, and a visible team footer. The landing page is meant to explain the product quickly and to give new users a clear path into sign-in or role-based usage.

Authentication is handled through a dedicated login and signup flow. The interface supports credential-based access and a student registration path, while role guards control where a user can go after login. This prevents users from landing in pages that do not match their role.

The dashboard side uses a shared shell with a sidebar, topbar, and content surface. The shell gives each role a consistent frame for navigation, while the pages inside it focus on domain work such as question management, exam creation, student attempts, user administration, and results review.

The UI pattern is intentionally utilitarian. Controls are grouped by task, state is surfaced clearly, and route-level shells keep the experience predictable. The code also uses reusable primitives so the same layout language appears across the public and authenticated sections.

3. Core Functional Areas

Public and authentication flow: the site provides landing, login, signup, and session handling. The authentication layer persists real users in Postgres and maps the authenticated identity into role-aware access.

Question bank and exam authoring: examiners can create objective questions, edit drafts, define sections, attach questions to exams, and check readiness before publishing.

Student assessment flow: students can discover assigned exams, open an attempt, navigate through questions, autosave answers, submit manually, or finish through timeout handling. The runtime is designed to preserve attempt state rather than treat the exam as a one-shot form.

Results and review flow: objective grading can be resolved automatically, while subjective or pending items move through a review workspace. Final results are aggregated only after readiness checks are satisfied.

Administration and reporting: admin-oriented pages cover user management, audit logs, result visibility, and reporting views so the system can be operated rather than just used.

The feature set is tied together by role-based navigation. A user does not just land on a generic dashboard; the interface is shaped by whether the person is a student, examiner, or admin, which reduces confusion and avoids exposing irrelevant actions.

4. Architecture and Implementation Style

The codebase uses a layered structure. The UI calls route handlers or server actions, those call service functions, and the services talk to the database or auth provider. This keeps the route files small and makes the business rules easier to test.

A shared database pool is used for persistence, and the auth flow is adapted through NextAuth credentials integration. The services use narrow interfaces and typed payloads so each function depends only on the data it really needs.

Validation is schema-first. Question and auth inputs are defined through typed contracts, which reduces mismatch between forms, API handlers, and domain logic. The same approach is used to separate ready, draft, pending review, and published result states.

The project also relies on composition in the frontend. Shared shells, fallback states, page wrappers, and role layouts are assembled from smaller primitives instead of being copied into each screen. That lowers duplication and keeps the page structure consistent.

5. Data Model and Persistence

The persistence layer is Postgres-backed and the project is structured around real entities rather than temporary state. Users, exams, questions, attempts, answers, results, and review records are persisted so the platform can survive reloads and deployments.

Authentication uses the database as the source of truth for account existence and role membership. This is important because login, signup, and route protection all need to agree on the same user state.

The student attempt flow is not a simple form submit. It carries attempt state across autosave, navigation, timeout, and final submission, which is why the repository keeps separate concepts for attempts, answers, and results.

6. Deployment, Quality, and Maintainability

The application is structured to be deployable on modern hosting platforms and to keep data and authentication real rather than mocked. The move to database-backed users and the role-based routing model make the project closer to a production assessment tool than a static demo.

The codebase also favors reusable UI primitives and shells so the public site and the dashboard remain visually consistent. This matters because the system contains several different operational screens, but they still need a single design language.

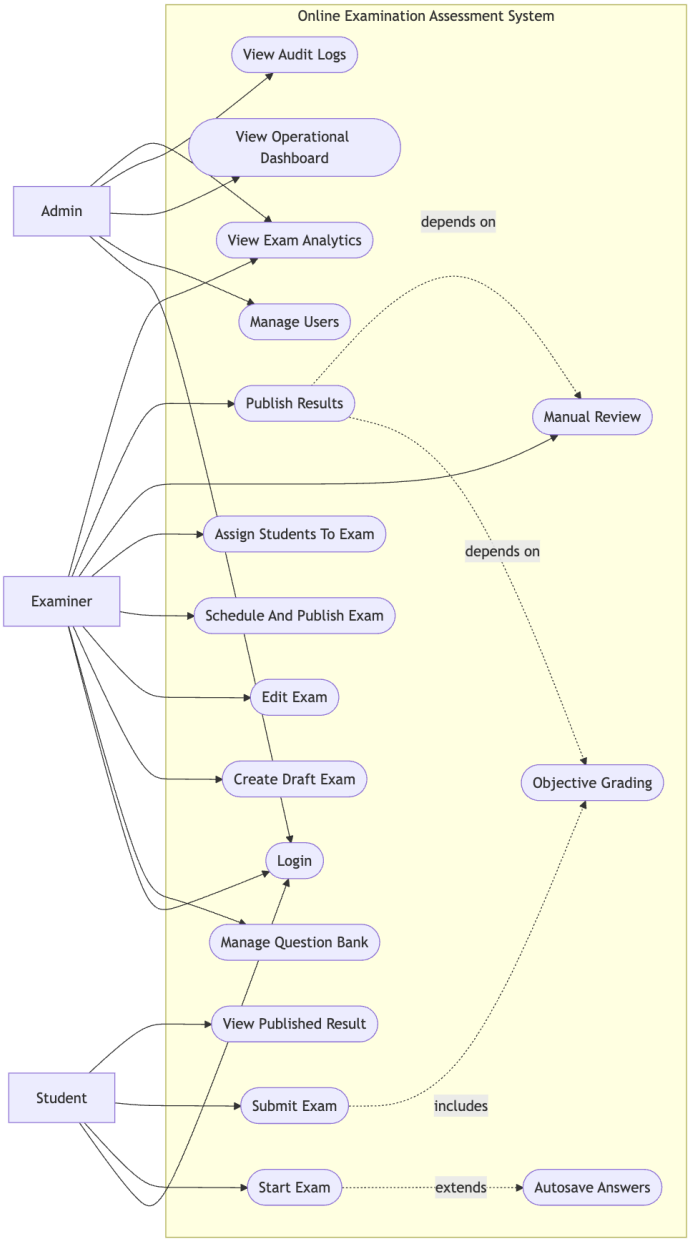
The repository also includes tests and schema validation around key flows, which helps the project stay stable as features move between auth, question authoring, grading, and reporting. That is the difference between a presentation-only demo and a maintainable application.

7. Diagram Walkthrough

The diagrams below capture the system from interaction, structure, runtime, and data perspectives. Each diagram is included with a short explanation so the report can stand on its own during a viva.

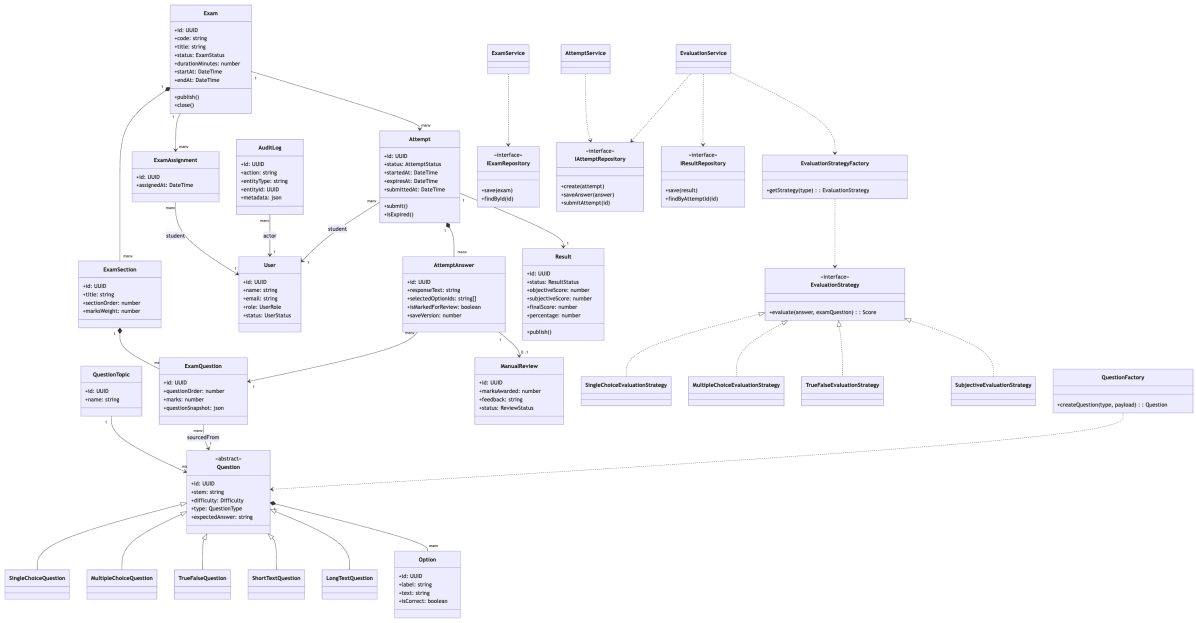
Use Case Diagram

Shows the three roles and their primary capabilities across the platform.



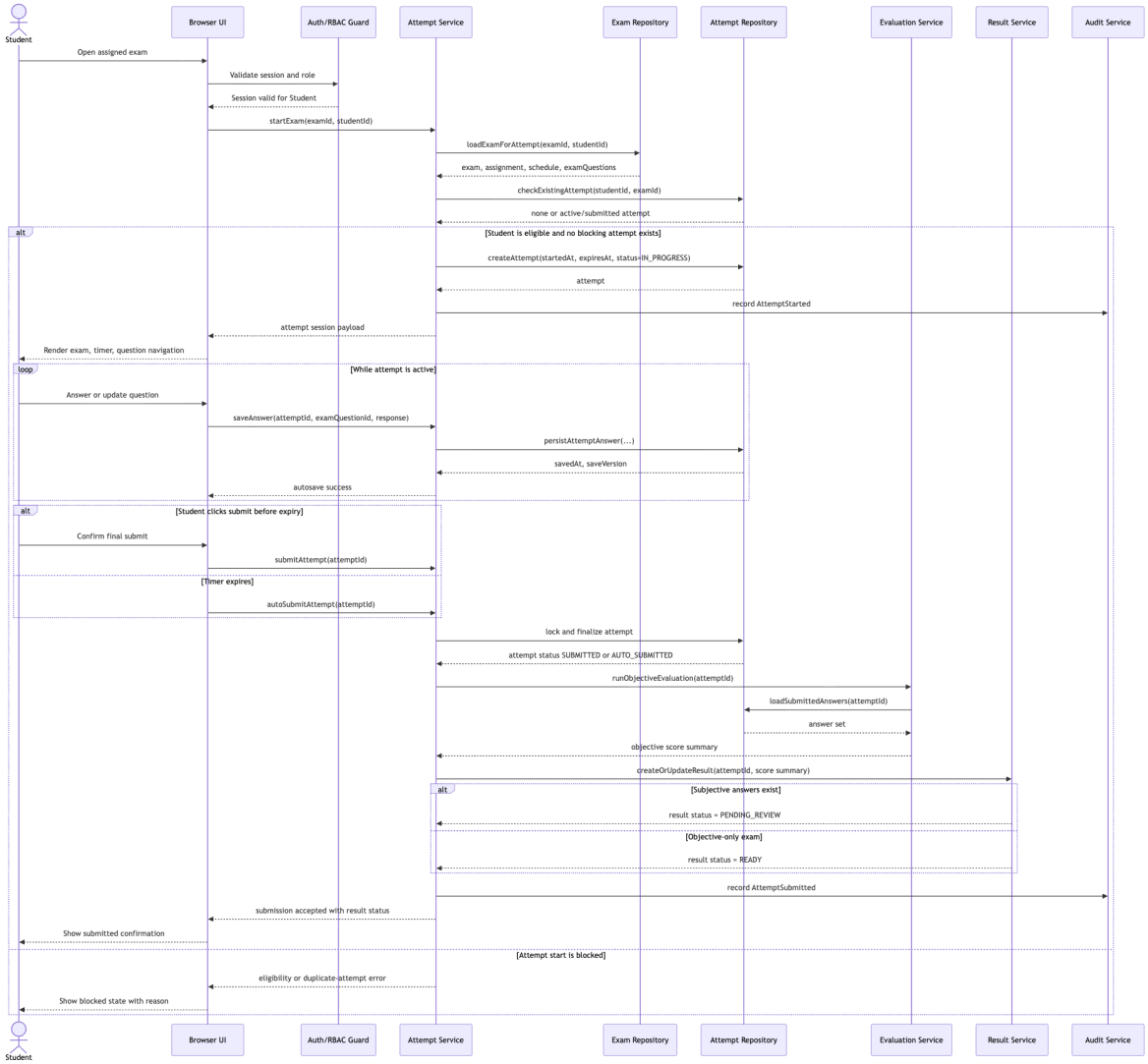
Class Diagram

Shows the static model of domain entities, services, and supporting abstractions.



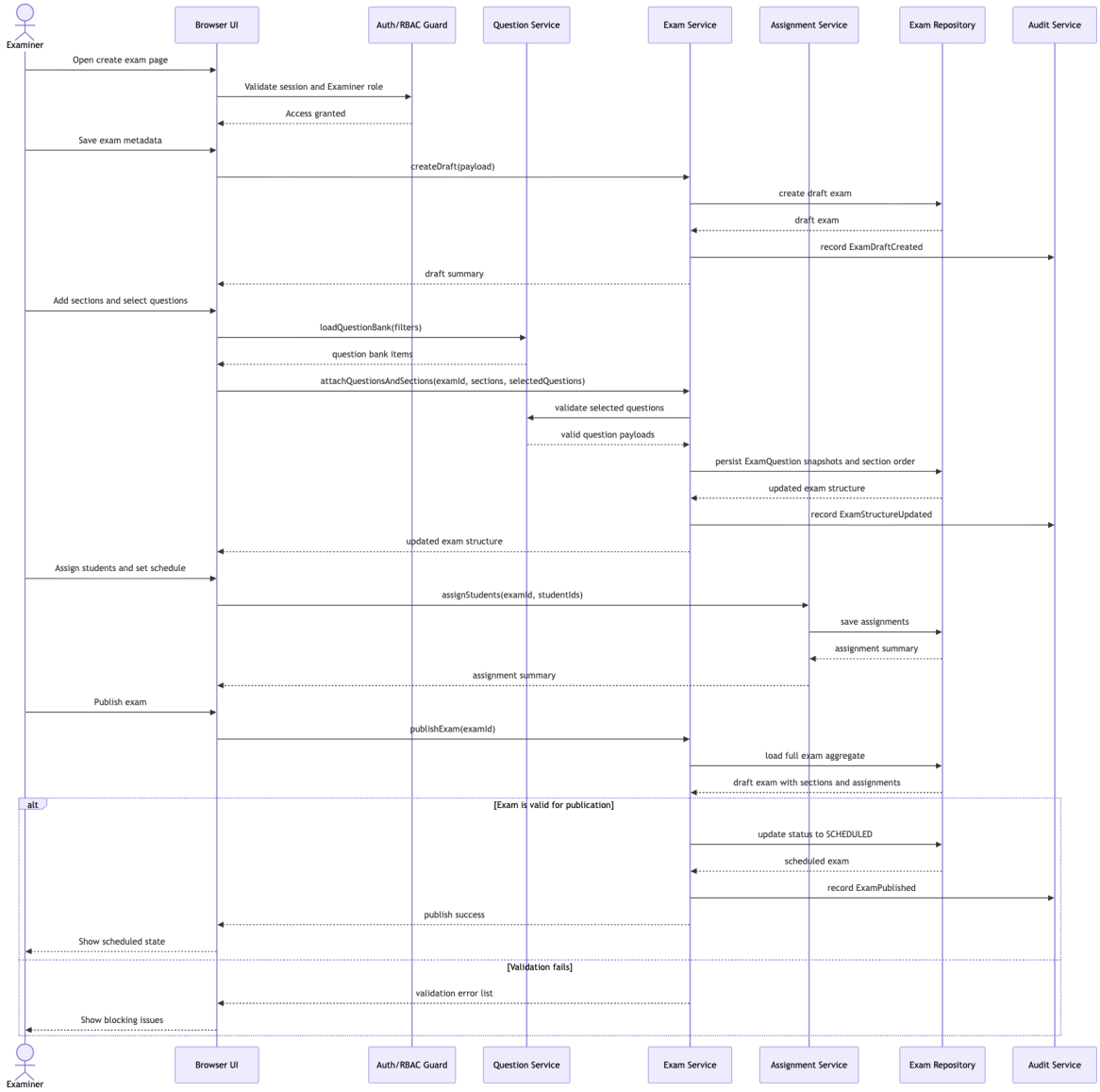
Sequence Diagram - Student Attempt

Shows the request path for opening, answering, autosaving, and submitting an attempt.



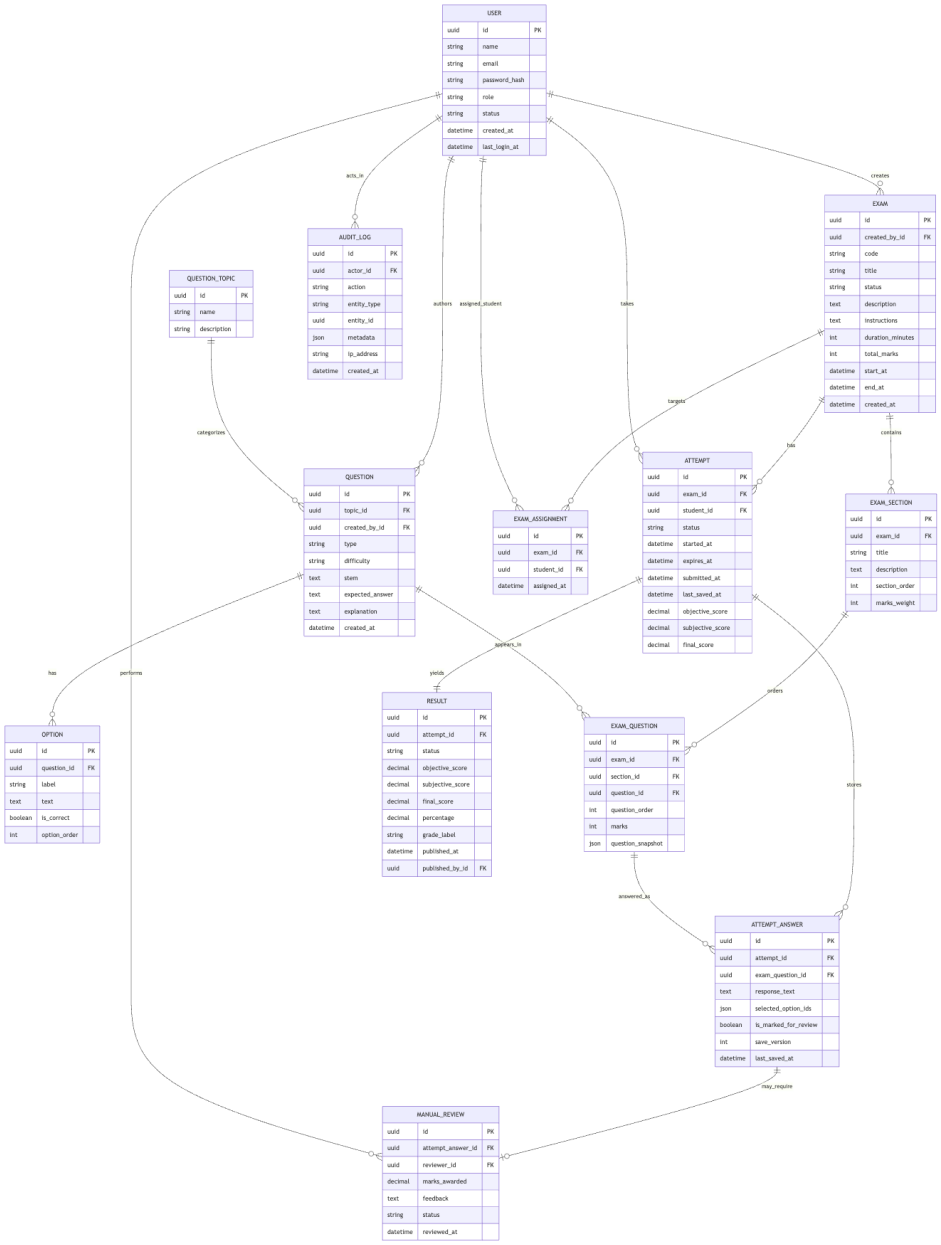
Sequence Diagram - Exam Creation

Shows the examiner flow from draft creation to scheduling, assignment, and publish readiness.



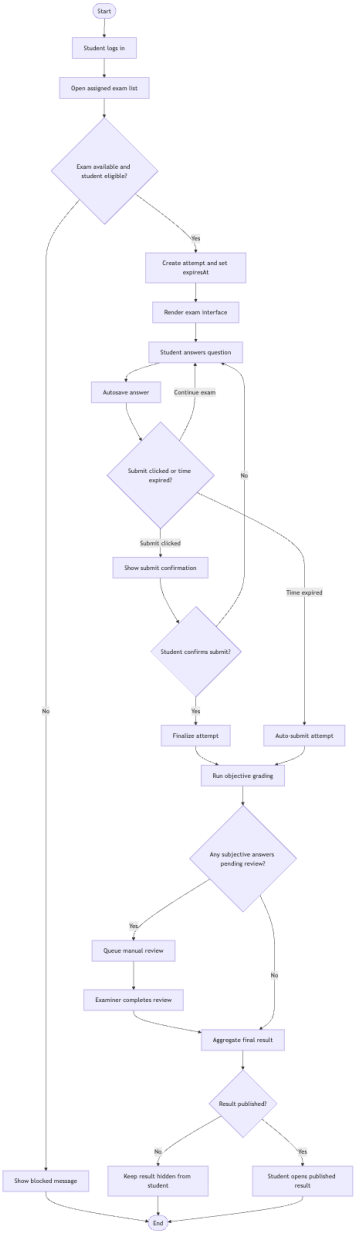
ER Diagram

Shows the database relationships that keep users, exams, attempts, answers, results, and review records connected.



Activity Diagram - Exam Flow

Shows the end-to-end exam lifecycle from eligibility checks to grading and publishing.



8. Contribution Matrix

The matrix below is qualitative and intentionally balanced. Every teammate is shown with the same structure so the report stays equal and does not rank people by count or score.

Teammate	Primary focus	Supporting work	Project value	Overall role
Yash Kumar	Repo shell, public UI, and shared layout	Auth flow, route guards, and deployment setup	Kept login and shell behavior consistent	Established the application foundation
Lakshyalamba	Baseline project structure and persistence	Student attempts, exam discovery, and storage	Linked exam and question management paths	Anchored the assessment engine
Rakshita	Question validation and authoring contracts	Question bank, exam configuration, and section flows	Objective and subjective question support	Made exam creation structured and complete
Ravleen Singh	Evaluation and result workflow	Grading strategies and review transitions	Aggregation, marks entry, and publish gating	Turned submissions into managed results
Abhishek Rana	Admin route structure and oversight pages	User management, audit logs, and filters	Reporting views and detail analysis	Gave the platform operational visibility

The matrix is intentionally balanced: each teammate is described in the same table structure, without numeric scoring, so the report reflects collaboration rather than ranking.